

Session 4 · Deck 4.B2

Quadrature: Turning an Expectation into a Sum

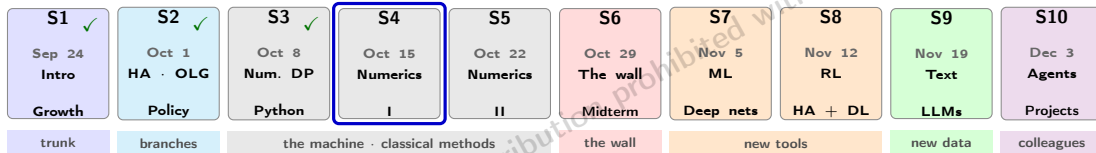
Zhigang Feng

Thursday, October 15, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 4.B2 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 4.B1 approximation and interpolation · 4.B2 quadrature · 4.B3 discretizing an AR(1) · 4.B4 fast and accurate · 4.B5 the worked example, end to end.

Seventeen Points, or a Million

Compute $\mathbb{E}[e^z]$ for $z \sim N(0, 1)$. The answer is $e^{1/2} = 1.64872127$.

rule	points	error
Gauss–Hermite	17	0
Gauss–Hermite	9	7×10^{-11}
trapezoid	17	9×10^{-9}
Simpson	17	8×10^{-3}
Monte Carlo	1 million	2×10^{-3}

Seventeen well-chosen points beat a million random ones by **thirteen orders of magnitude**.

Why you should care

Every Bellman sweep evaluates

$$\mathbb{E}[\hat{v}(k', z') \mid z]$$

at every state. It is the inner sum in every solve we have run.

And notice the row that is *worse* than the one above it. Simpson is a higher-order rule than the trapezoid rule — and here it loses by six decades. That is the lesson of this deck.

The Object, and the Two Choices

Every rule has the same shape — evaluate at nodes, add up with weights:

$$\int f(z) w(z) dz \approx \sum_{m=1}^n \omega_m f(z_m).$$

So there are exactly two choices: **where the nodes go** and **what the weights are**. Everything else is a consequence.

The rules divide by how they make those choices:

- **Fix the nodes** (equally spaced) and solve for weights: Newton–Cotes.
- **Choose both**: Gaussian rules, which buys twice the accuracy for the same n .
- **Draw the nodes at random**: Monte Carlo.

w is the weight, and it matters

For an expectation, w is the **density**. A rule that knows about w can place nodes where the probability is, instead of spreading them evenly over a region that is mostly empty.

That single observation explains the whole table on the previous slide.

Newton–Cotes: Equally Spaced, and the Surprise

Fit a polynomial through equally spaced nodes and integrate it exactly.

- **Trapezoid** (degree 1): error $O(h^2)$.
- **Simpson** (degree 2): error $O(h^4)$.

n	trapezoid	Simpson
5	2.4×10^{-2}	5.5×10^{-1}
9	2.4×10^{-2}	2.4×10^{-2}
17	8.8×10^{-9}	7.9×10^{-3}
33	4.0×10^{-12}	2.9×10^{-9}

Measured on $\mathbb{E}[e^z]$, $z \sim N(0, 1)$, truncated at ± 8 standard deviations.

The higher-order rule loses

Simpson is $O(h^4)$ and trapezoid is $O(h^2)$, yet the trapezoid rule is a thousand times better at $n = 17$.

Why: for a smooth integrand whose derivatives vanish at both ends — which a normal density does, very fast — the Euler–Maclaurin error terms all cancel, and the trapezoid rule becomes *spectrally* accurate. The $O(h^2)$ bound is true and enormously pessimistic.

Gaussian Rules: Choose the Nodes Too

With n nodes and n weights you have $2n$ free numbers, so you can ask the rule to be **exact for every polynomial of degree up to $2n - 1$** — twice what Newton–Cotes achieves with the same count.

The nodes that do it are the roots of the orthogonal polynomials for the weight w :

- $w = 1$ on $[-1, 1]$: Legendre $\Rightarrow$ **Gauss–Legendre**.
- $w = e^{-z^2}$ on $\mathbb{R}$: Hermite $\Rightarrow$ **Gauss–Hermite**, which is the normal density.
- $w = e^{-z}$ on $[0, \infty)$: Laguerre.

For $z \sim N(\mu, \sigma^2)$, substitute $z = \mu + \sqrt{2}\sigma x$ and the Hermite nodes and weights apply directly.

n	Gauss–Leg.	Gauss–Herm.
3	1.2	1.1×10^{-2}
5	1.8×10^{-1}	4.2×10^{-5}
9	1.3×10^{-1}	7.2×10^{-11}
17	2.8×10^{-4}	0

Both are Gaussian rules. The difference is entirely that **Hermite knows the weight is a normal density** and Legendre does not — it is spending its nodes on a truncated interval that is mostly empty.

Monte Carlo: Slow, and Indifferent to Dimension

Draw $z_1, \dots, z_n$ from the distribution and average:

$$\mathbb{E}[f(z)] \approx \frac{1}{n} \sum_m f(z_m), \quad \text{s.e.} = \frac{\sigma_f}{\sqrt{n}}.$$

n	error	s.e.
100	4.0×10^{-2}	1.7×10^{-1}
10,000	8.9×10^{-3}	2.0×10^{-2}
1,000,000	2.4×10^{-3}	2.2×10^{-3}

A hundredfold increase in draws buys one decimal digit. That is the $n^{-1/2}$ rate, and no amount of cleverness changes the exponent.

And yet it wins, eventually

$n^{-1/2}$ **does not depend on the dimension.** A tensor Gauss rule needs n^d nodes; Monte Carlo needs the same n whether d is 1 or 100.

So the crossover is a property of d , not of taste: below about five dimensions use a Gaussian rule, far above it you have no choice — which is the regime Sessions 7 and 8 live in.

Quasi-Monte Carlo (Sobol, Halton) replaces randomness with a low-discrepancy sequence and recovers close to n^{-1} in moderate dimensions.

Choosing a Rule

Rule	Exact for	Use it when
Trapezoid	degree 1, but spectral on a smooth decaying integrand	a fast-decaying integrand on a wide interval
Simpson	degree 3	rarely — not the safe default people assume
Gauss–Legendre	degree $2n - 1$ on $[a, b]$	a genuinely bounded domain
Gauss–Hermite	degree $2n - 1$ against a normal weight	normal shocks — the macro default
Monomial rules	low degree, few nodes	moderate dimension
Monte Carlo / QMC	nothing exactly	high dimension, or sampling only

The rule should know the weight. Gauss–Hermite was exact at seventeen nodes and Gauss–Legendre was not, on the same integral, with the same node count — and the only difference was that one of them was built for a normal density.

Takeaways

- Every quadrature rule is **nodes and weights**. The only questions are where the nodes go and whether the rule knows the density.
- A Gaussian rule with n nodes is exact for polynomials up to degree $2n - 1$ — twice what equally spaced nodes deliver for the same cost.
- **Gauss–Hermite was exact at 17 nodes**. Gauss–Legendre, with the same 17, was off by 3×10^{-4} ; a million Monte Carlo draws were off by 2×10^{-3} .
- Higher order is not the same as more accurate: **Simpson lost to the trapezoid rule by a thousand times** at $n = 17$, because on a smooth decaying integrand the trapezoid rule is secretly spectral.
- Monte Carlo converges at $n^{-1/2}$ **regardless of dimension**. That is terrible in one dimension and the only option in fifty.

Bridge: Nodes and Weights, or a Markov Chain?

Gauss–Hermite gives an excellent approximation to $\mathbb{E}[f(z')]$ when z' is drawn afresh each period. But in every model we have solved, z is **persistent**:

$$\ln z' = \rho \ln z + \sigma \varepsilon',$$

so the distribution of z' depends on today's z . What we have been using is not a quadrature rule at all — it is a **finite Markov chain**, a grid plus a transition matrix Π .

We have written Π down by hand in Session 3 and again this morning. Where does it actually come from?

Next (4.B3)

Two standard constructions, Tauchen and Rouwenhorst — and a measurement of how badly they disagree when ρ is close to one, which is the range macroeconomics actually uses.

Session 3 promised you this comparison. It is more one-sided than you might expect.